

BrowserAgent General Autonomous Agent Architecture

Corrected Research Synthesis + Claude-Code-Ready Migration Specification

*Repository baseline: Reshwant-Borra/BrowserAgent main @ 58a7d5a095900be07614e2992d5c14b72732036c
(28 Aug 2026)*

Prepared as an implementation specification for Claude Code; research only, no production code applied.

Executive Decision

BrowserAgent should evolve into a general controller that dynamically creates and revises subgoals, while continuing to use the existing AgentLoop, browser backend, verifier, event store, memory, BatchOrchestrator, WorkflowOrchestrator, research discovery, and UI/trace infrastructure as deterministic capabilities. The central change is task-level control, not a replacement of the execution engine.

Most important correction to the prior research PDF: Do not create a parallel src/ architecture, do not rebuild the action executor, do not delete BatchOrchestrator or WorkflowOrchestrator, and do not replace the current PageObservation with BrowserGym AXTree before an ablation proves a need. The current repository already contains structured action primitives, per-observation numeric element IDs, deterministic target re-resolution, verification, event sourcing, and durable orchestration.

Decision	Verdict	Implementation implication
General Task Workspace	YES	Add an event-backed workspace projection; reuse existing task_state and active_task_facts where possible.
Event sourcing canonical	YES	Existing memory/events table remains truth; workspace is a rebuildable projection.
Dynamic subgoals	YES	General controller owns high-level subgoals; AgentLoop executes the active subgoal.
Dynamic replanning	YES, bounded	Replan only on explicit triggers: subgoal completion, resource discovery, repeated failure, contradiction, user update, or completion rejection.
Procedural skills	LATER	Only after general-controller holdouts pass without skills. Skills are declarative hints/macros, never privileged code.
Site-specific skills	LATER / restricted	Allow only if validated and scoped; never required for baseline generality.
Arbitrary Python	NO for browser control	Optional future safe computation tool may expose predefined operations; no host-code generation.
Arbitrary JavaScript	NO	Do not make model-generated page scripts a normal capability.
Separate planner model	NO now	Use qwen3:8b with distinct structured planner/executor prompts; benchmark before changing models.
Vision	FALLBACK later	DOM/accessibility-first remains default. Add vision only after a measured

		observation-insufficiency gate.
Embeddings	NOT NEEDED now	Current FTSS is proven. Add semantic retrieval only if skill/memory retrieval ablations require it.
Parallelism	LATER	Keep sequential baseline; existing batch is the first delegation surface for independent work.
BatchOrchestrator	KEEP + EXTEND	Expose as planner-callable delegate for repeated independent targets; do not replace it with a serial planner loop.
WorkflowOrchestrator	KEEP + EXTEND initially	Use as compatibility/verified ordered delegate. General controller may supersede some use cases later, after benchmarks.
Research subsystem	KEEP + REFACTOR as capability	Discovery + evidence gathering becomes a callable capability, not the only top-level research mode.
Semantic router	REFACTOR / compatibility	Keep resource-resolution strengths and legacy mode while general controller takes over task-level planning.
Qwen3-8B	PARTIALLY SUFFICIENT overall	Already sufficient for executor and semantic routing; planner reliability must be measured on new generality holdouts.

1. Scope and Corrections to the Original Research

The 21-page migration research identified the correct strategic direction: declarative model intent, deterministic execution, a generic workspace, hierarchical planning, bounded context, and prompt-injection defenses. This revision preserves those conclusions where supported, but corrects factual and repository-grounding errors before Claude Code is allowed to implement anything.

1.1 Corrections to external-product claims

- OpenAI Operator was powered by Computer-Using Agent (CUA), which uses screenshots plus mouse/keyboard-style GUI actions. This is documented by OpenAI.
- Operator functionality was integrated into ChatGPT agent in July 2025. It was not “absorbed into Atlas.” Atlas launched later, in October 2025, and was subsequently deprecated in 2026 as browser-based agentic capabilities moved into ChatGPT/Codex experiences.
- OpenAI publicly documented that ChatGPT agent combined a visual browser, text-based browser, terminal, APIs/connectors, and user takeover. The exact internal planner/memory architecture is proprietary and must not be inferred as fact.
- Perplexity publicly documents Comet as Chromium-based, with an Assistant side panel that can use current-tab or explicitly referenced-tab context, perform browser control, and honor domain-specific

Browser Control / Read Only / No Access permissions. Reverse-engineered extension internals are not treated as official architecture facts.

- Anthropic publicly describes computer use as screenshot/GUI interaction and explicitly identifies prompt injection as a major browser-agent threat. Its 2025 prompt-injection defense work also reports model training, classifiers, and red-team defenses; no browser-agent system should treat prompt injection as solved.

1.2 Corrections to repository assumptions

- The current repository does not use a src/ package layout. New code must live under existing top-level packages such as agent/, memory/, browser/, router/, batch/, workflow/, research/, ui/, and cli/.
- BrowserAgent already has a deterministic execution engine. agent/schemas.py defines typed browser actions; agent/loop.py executes and verifies them; browser/playwright_backend.py is the only live browser backend. Phase 2 of the original report would have duplicated this proven machinery.
- BrowserAgent already has per-observation numeric element IDs. browser/page_model.py states that IDs are intentionally ephemeral and re-resolved from the current observation; browser/observer.py enumerates interactive controls and assigns IDs. BrowserGym-style bids are therefore an optional observation experiment, not a prerequisite.
- The validated model context window in config is 8192, not 128K. Architectural decisions must use the configured, proven runtime, not a theoretical model maximum.
- BatchOrchestrator and WorkflowOrchestrator are durable, verified execution substrates, not obsolete RPA stubs. They both reuse AgentLoop rather than creating competing browser engines.

2. Current Repository Ground Truth

This section is intentionally code-oriented. Claude Code should treat these current files as the integration points and preserve their invariants.

Subsystem	Current implementation	Invariant to preserve
Browser actions	agent/schemas.py + agent/decision.py	GBNF/JSON/Pydantic/semantic validation before execution; no arbitrary selector generation.
Core execution	agent/loop.py	Observe -> model decision -> validate -> risk/approval -> execute -> re-observe -> verify -> event log -> recovery.
Observation	browser/observer.py + browser/page_model.py	Compact DOM/accessibility-like representation; numeric IDs valid only for one observation; live element re-resolution before action.
Browser lifecycle	browser/playwright_backend.py	launch and cdp_attach; persistent user browser remains independent from agent process.
Verification	agent/verifier.py	Deterministic/hybrid browser-state verification; no second LLM required for ordinary action verification.
Task state	memory/event_store.py + memory/replay.py + memory/task_state.py	Append-only events are canonical; TaskState is a replayable projection.

Memory	memory/schema.sql + task_memory.py + context_builder.py	Running summary + FTSS task memories + active facts under a bounded prompt budget.
Semantic planning	router/semantic_planner.py + plan_schema.py	Model says what the user means; deterministic ResourceResolver says what resources actually exist.
Batch	batch/orchestrator.py + batch/store.py	Durable independent work-item ledger, child AgentLoops, crash/retry reconciliation, structured results.
Ordered workflow	workflow/orchestrator.py + workflow/store.py	Persisted ordering, verified-before-advance, cross-step fact passing into the same active-fact machinery.
Research	research/discovery.py + BatchOrchestrator	Deterministic link enumeration; model chooses candidate IDs rather than inventing URLs.
UI/diagnostics	ui/jobs.py + ui/store.py + cli/trace.py	Durable UI jobs; stop/clarification/approval; forensic reconstruction from persisted data.

2.1 Existing primitives already satisfy the “safe machinery” requirement

The current ModelAction union already covers open_url, click, type, select, scroll, back, extract, download, wait, and finish. The desired redesign should add task-control/workspace capabilities around these primitives rather than replace them. The general agent “creates its own actions” by composing these primitives into novel procedures and by creating/revising subgoals and workspace objects; it does not gain arbitrary Playwright, Python, or JavaScript authority.

3. Lessons from Current External Systems

System	Documented architecture/capability lesson	What BrowserAgent should copy	What not to infer/copy
OpenAI Operator / CUA	CUA used screenshot-based GUI perception and mouse/keyboard-like interaction; self-correction and user handoff were explicit product concepts.	General UI compatibility, takeover/approval boundaries, recovery to user.	Do not assume screenshot-first is optimal for a local 8B system; internal planner and memory are proprietary.
ChatGPT agent (2025) / current browser-agent lineage	OpenAI documented a unified agent choosing among visual browser, text browser, terminal, APIs/connectors; user could interrupt, steer, take over, and receive partial progress.	A general controller choosing among capabilities is stronger than choosing one workflow type up front.	Do not copy terminal/code authority into BrowserAgent by default; the local product has different risk/compute constraints.
Perplexity Comet	Chromium browser + Assistant side panel; context from current tab or @tab; browser actions; domain-specific Browser Control/Read Only/No Access;	Persistent in-browser context, explicit tab identity, domain capability policies, minimal-context extraction.	Do not treat reverse-engineered extension internals as official planner/memory facts.

	local-vs-sent context controls.		
Anthropic computer use	Screenshot/coordinate computer use; strong warning that every webpage can carry prompt injection; later defenses combine training, classifiers, and red teaming.	Treat web content as untrusted; add security tests and layered controls independent of planner compliance.	Do not move to pixel-only control unless DOM failure data justifies it.
BrowserGym / AgentLab	Well-defined observation/action spaces, DOM + AXTree + screenshot options, unique element IDs, open-tab observations, standardized multi-benchmark evaluation.	Adopt benchmark discipline and optional observation experiments; keep stable ID grounding.	Do not rewrite the observer just to match BrowserGym; BrowserAgent already has analogous numeric IDs and Playwright/CDP.
Plan-and-Act	Long-horizon results support separating high-level planning from low-level execution.	Use low-frequency structured planning plus high-frequency focused execution.	Do not assume a second model is required; same local model can fill two contracts initially.

4. Target Architecture

The target is a two-level controller built around the existing execution and persistence stack. The general controller owns task-level intent, subgoals, workspace state, delegation, and completion. Existing AgentLoop remains the browser-action executor. Existing orchestrators become optional delegates selected by the controller when their execution structure is advantageous.

USER GOAL

|

v

GeneralAgentController (new, low-frequency control)

| -- load/rebuild TaskWorkspace from events

| -- resolve resources using existing resolver

| -- create/revise subgoals via structured Qwen call

| -- choose execution substrate

|

+--> Direct subgoal -> existing AgentLoop

+--> Independent N targets -> existing BatchOrchestrator

+--> Verified fixed sequence -> existing WorkflowOrchestrator (compatibility/optimization)

+--> Need sources -> existing research discovery + batch

|

v

Workspace ingest + deterministic verification/computation

|

+--> continue current plan

+--> bounded replan

+--> ask user

+--> completion evaluation

v

FINAL RESULT

4.1 Why this is general rather than workflow-bound

Task modes are no longer the user-facing ontology. The controller may discover at runtime that a subproblem is parallelizable, sequential, research-oriented, or a direct interaction. Those are execution strategies, not task categories. The controller can also alternate strategies during one task. For example, a shopping task can use discovery, delegate candidate collection to batch, perform deterministic comparison in the workspace, and then open the selected item in a direct AgentLoop.

5. What “Create Its Own Actions” Means

Level	Allowed?	Policy
Choose next primitive	YES	Model chooses among validated browser primitives from current observation.
Compose novel multi-step procedure	YES	Planner creates/revises subgoals at runtime.
Create arbitrary task-local entities/facts	YES	Structured workspace mutations with provenance.
Create declarative reusable skill	LATER	Skill may propose a sequence/template but still runs through normal validation/verification.
Generate JavaScript for page execution	NO	Too much authority and injection surface; not a baseline capability.
Generate host Python for browser control	NO	Would bypass the deterministic boundary.
Self-modify production source	NO	Code changes remain a developer workflow, never an autonomous runtime behavior.

6. TaskWorkspace: Minimal Event-Backed Design

The prior report proposed one giant SQLite JSON document. This revision recommends a hybrid projection because BrowserAgent already has normalized task metadata, active facts, and event sourcing. The new workspace should add only what the current schema cannot represent cleanly: arbitrary entities, evidence links, open questions/hypotheses, and completion requirements.

6.1 Canonical vs derived state

State class	Storage	Canonical?	Notes
User goal / immutable task record	existing tasks table	Yes	Current EventStore.create_task.
Action/verification history	existing events table	Yes	Never overwritten.
Current subgoal / plan	existing task_state projection +	Derived	Already present; extend

	SUBGOAL_CHANGED		semantics instead of replacing.
High-confidence scalar facts	existing active_task_facts	Derived from events	Reuse for exact values/constraints that action guard can apply.
Flexible workspace metadata	new workspace_state JSON projection	Derived	Open questions, completion requirements, hypotheses, resource refs, controller metadata.
Arbitrary entities	new workspace_entities projection	Derived	Generic entity_type + name + JSON attributes + status.
Evidence/provenance	new workspace_evidence projection	Derived	Links entity/fact/workspace field to source event/URL/excerpt/verification.

6.2 Proposed schema migration

-- Append to memory/schema.sql (projection tables only; events remain truth)

```
CREATE TABLE IF NOT EXISTS workspace_state (
  task_id TEXT PRIMARY KEY,
  version INTEGER NOT NULL DEFAULT 1,
  data TEXT NOT NULL,          -- JSON object
  last_event_id INTEGER,
  updated_at TEXT NOT NULL,
  FOREIGN KEY(task_id) REFERENCES tasks(id)
);
```

```
CREATE TABLE IF NOT EXISTS workspace_entities (
  id TEXT PRIMARY KEY,
  task_id TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  name TEXT,
  attributes TEXT NOT NULL,    -- JSON object
  status TEXT NOT NULL DEFAULT 'active',
  source_event_id INTEGER NOT NULL,
  updated_event_id INTEGER NOT NULL,
  FOREIGN KEY(task_id) REFERENCES tasks(id)
);
CREATE INDEX IF NOT EXISTS idx_workspace_entities_task
ON workspace_entities(task_id, entity_type, status);
```

```
CREATE TABLE IF NOT EXISTS workspace_evidence (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  task_id TEXT NOT NULL,
  entity_id TEXT,
  fact_id INTEGER,
  field_key TEXT,
  source_event_id INTEGER NOT NULL,
  source_url TEXT,
  excerpt TEXT NOT NULL,
```

```

confidence REAL NOT NULL DEFAULT 1.0,
verification_status TEXT NOT NULL DEFAULT 'observed',
created_at TEXT NOT NULL,
FOREIGN KEY(task_id) REFERENCES tasks(id)
);
CREATE INDEX IF NOT EXISTS idx_workspace_evidence_task
ON workspace_evidence(task_id, source_event_id);

```

6.3 New Python contracts

```

# New: agent/workspace_models.py
class WorkspaceEntity(BaseModel):
    id: str
    entity_type: str
    name: str | None = None
    attributes: dict[str, JsonValue] = Field(default_factory=dict)
    status: Literal['active', 'rejected', 'selected', 'resolved'] = 'active'

```

```

class EvidenceRef(BaseModel):
    entity_id: str | None = None
    fact_key: str | None = None
    field_key: str | None = None
    source_event_id: int
    source_url: str | None = None
    excerpt: str
    confidence: float = Field(ge=0, le=1)

```

```

class WorkspacePatch(BaseModel):
    add_entities: list[WorkspaceEntity] = []
    update_entities: list[WorkspaceEntityPatch] = []
    add_facts: list[WorkspaceFact] = []
    add_evidence: list[EvidenceRef] = []
    open_questions_add: list[str] = []
    open_questions_resolve: list[str] = []
    completion_requirements: list[str] | None = None

```

6.4 Mutation rule

The model never writes SQLite. It emits `WorkspacePatch`. Deterministic code validates it, checks referenced event/entity IDs, appends one `WORKSPACE_MUTATED` event containing the validated patch, and updates projection tables in the same controller transaction. Rebuild logic can recreate projections from events. This mirrors the existing event-sourcing invariant instead of adding a second truth store.

7. Planner and Controller Contracts

The existing router `TaskPlan` is useful but tied to fixed `execution_shape`. Do not delete it immediately. Introduce a controller contract that expresses the next high-level decision without committing the entire task to one execution mode.

```

# New: agent/controller_models.py
class ControllerDecision(BaseModel):
    decision: Literal[
        'start_subgoal',

```

```

        'delegate_batch',
        'delegate_workflow',
        'discover_sources',
        'ask_user',
        'revise_plan',
        'finish'
    ]
    reason_code: Literal[
        'initial_plan', 'subgoal_complete', 'new_constraint', 'repeated_failure',
        'resource_missing', 'independent_targets', 'ordered_dependency',
        'evidence_insufficient', 'completion_satisfied', 'user_update'
    ]
    active_subgoal: str | None = None
    plan: list[str] | None = None
    resource_refs: list[int] = []
    clarification_question: str | None = None
    completion_claim: str | None = None

class CompletionEvaluation(BaseModel):
    satisfied: bool
    missing_requirements: list[str] = []
    unsupported_claims: list[str] = []
    next_recommendation: Literal['finish', 'continue', 'replan', 'ask_user']

```

7.1 Receding-horizon control policy

1. Initial planner call: derive 2-5 high-level subgoals, completion requirements, and resource needs. Do not plan clicks.
2. Execute the current subgoal using the existing AgentLoop action loop.
3. Persist useful facts/entities/evidence as structured workspace mutations.
4. Replan only when a trigger fires: subgoal complete, repeated recovery failure, new constraint, missing resource, contradiction, user clarification, or final-completion rejection.
5. Use deterministic computation for sortable/computable attributes once entities are structured.
6. Call completion evaluation only at natural boundaries, not after every action.

7.2 Controller pseudocode

```

async def run_general_task(task_id):
    workspace = workspace_store.rebuild_if_stale(task_id)

    if not workspace.plan:
        decision = await planner.initial_plan(goal, available_resources, workspace.summary())
        apply_controller_decision(decision)

    while not terminal(task_id):
        state = load_parent_state(task_id)
        workspace = workspace_store.load(task_id)

        if needs_user_input(workspace):
            return WAITING_FOR_INPUT

        if should_replan(state, workspace):
            decision = await planner.replan(bounded_controller_context(...))

```

```

    apply_controller_decision(decision)
    continue

    subgoal = workspace.current_subgoal
    substrate = choose_deterministic_substrate(subgoal, workspace)

    result = await execute_via_existing_substrate(substrate, subgoal, workspace)
    ingest_verified_result_as_workspace_events(result)

    if boundary_reached(result):
        completion = await completion_evaluator(goal, workspace.completion_view())
        persist_completion_evaluation(completion)
        if completion.satisfied:
            finalize_from_workspace()
            return COMPLETED

```

8. Execution Substrates: Keep the Proven Machinery

Substrate	When controller should choose it	Required change
AgentLoop	One page/site or one bounded interactive subgoal.	Add general-mode parent/task metadata and a structured subgoal result contract; otherwise preserve execution.
BatchOrchestrator	N independent targets that can use the same child objective/result contract.	Allow parent controller/workspace IDs and generic entity-result ingestion. Keep retry/crash/dedupe ledger.
WorkflowOrchestrator	A fixed, already-resolved ordered chain where deterministic verify-before-advance is valuable.	Expose as delegate; allow parent workspace facts to seed steps and ingest outputs back. Do not delete until general controller proves equal reliability.
research/discovery.py	Planner needs candidate web sources and has no explicit resource.	Expose discovery as a capability returning canonical candidate resources; preserve ID selection/no URL hallucination.

8.1 Deterministic substrate selection before using the LLM

Some delegation choices can be mechanical after the planner has produced a structured subgoal. If a subgoal contains 20 resolved independent URLs with one result schema, BatchOrchestrator is the obvious substrate. If it contains one page, use AgentLoop. Only ambiguous structural choices require a controller model call. This keeps model-call count bounded.

9. Generic Entity Collection and Comparison

The Amazon failure demonstrated the missing abstraction: BrowserAgent could extract several values but could not maintain a generic set of comparable candidates. The fix is not a ProductRanker. It is workspace entities plus deterministic query/computation primitives.

workspace entity example (model-defined type; schema-generic):

```
{
  "id": "ent_004",
  "entity_type": "candidate",
  "name": "Example Vacuum",
  "attributes": {
    "price_usd": 159.99,
    "rating": 4.6,
    "review_count": 12000,
    "features": ["cordless", "HEPA"]
  }
}
```

9.1 Safe computation surface

Do not let Qwen write Python. Add deterministic workspace operations owned by code, for example: filter_entities(predicate_spec), sort_entities(field, direction), numeric_min/max, date_min/max, dedupe_entities(keys), count, group_by, and select_top_k after the planner specifies comparison criteria. Semantic ranking remains an LLM task, but its inputs are structured candidate records and its output must reference entity IDs.

9.2 Ranking contract

```
class RankRequest(BaseModel):
    entity_ids: list[str]
    objective: str
    hard_constraints: list[str] = []
    numeric_preferences: list[NumericPreference] = []
    semantic_preferences: list[str] = []
    k: int = Field(ge=1, le=20)

class RankResult(BaseModel):
    ranked_entity_ids: list[str]
    rationale_by_entity: dict[str, str]
    missing_information: list[str] = []
```

The controller may decide to collect more evidence when missing_information affects the requested ranking. This is how “find the three best” becomes a general procedure applicable to products, colleges, hotels, internships, papers, or assignments.

10. Observation Architecture: Do Not Rewrite Yet

BrowserGym shows that DOM, AXTree, screenshot, stable element IDs, and open-tab metadata are useful observation options. BrowserAgent already has a compact interactive-element view with numeric IDs and a deterministic selector/nth re-resolution invariant. Therefore observation replacement is not Phase 1.

Question	Current evidence	Decision
Need unique grounded element IDs?	Already implemented in PageObservation; IDs intentionally scoped to each observation.	KEEP.
Need Accessibility Tree immediately?	No failure evidence yet proving current	ABLATe later.

	observer is the bottleneck; Amazon extraction succeeded.	
Need page regions/cards?	Potentially useful for multi-entity extraction and dynamic pages.	Add optional semantic grouping experiment after workspace/controller baseline.
Need screenshot vision?	Visual-only/canvas tasks remain an unvalidated gap.	Fallback later, gated by observed failures.
Need page deltas?	Phase 4 previously found no need; dynamic hash churn was fixed in loop detection.	Do not build without new evidence.

11. Memory Architecture

Layer	Purpose	Implementation
Working memory	What Qwen needs for the current decision.	Existing context_builder + current subgoal + relevant entity snippets + recent actions + current observation.
Task/episodic memory	Durable history and task state.	Existing events, summaries, task_memories, active_task_facts + new workspace projections.
Cross-task memory	Reusable stable knowledge/preferences when explicitly appropriate.	Not required for first migration phases. Introduce only with separate provenance/scope policy; do not silently reuse arbitrary prior task facts.
Procedural skill memory	Reusable procedures that reduce planning/action cost.	Later experiment after generality baseline. Store declarative, versioned skills with success/failure stats; retrieve as planner hints, never privileged execution.

11.1 Skill lifecycle - later only

- Observe repeated successful procedure across at least two materially different tasks or repeated site interactions.
- LLM proposes a declarative skill template referencing only existing capabilities and parameters.
- Static validator rejects unknown actions, unsafe domains, missing parameter types, or embedded executable code.
- Skill is stored as candidate, not promoted.
- Run holdout validation; compare success, steps, latency, and negative transfer against no-skill baseline.
- Promote only after a predefined success threshold; deprecate automatically on repeated verified failures.

12. Security Architecture: Zero Trust for Page Content

Prompt injection is the largest architectural risk introduced by general autonomy. OpenAI, Anthropic, and independent AgentDojo/ST-WebAgentBench research all show that model-level instruction-following alone is insufficient. Security controls must constrain what a compromised planner can do.

Boundary	Required mechanism
Instruction trust	System/user instructions are authoritative. Page text, documents, search results, ads, and tool outputs are explicitly untrusted data.
Capability scope	Planner cannot invent tools. Every operation is a typed capability with deterministic validation.
Data-flow policy	Cross-origin transfer of sensitive values requires explicit provenance and policy; never allow arbitrary “read from A, type into B” when data is marked sensitive.
Risk gate	Retain classify_risk + human approval for consequential actions; expand from keyword-only to structured action intent/context over time.
Domain policy	Add per-task/domain Browser Control / Read Only / No Access policy inspired by Comet enterprise controls.
Sensitive inputs	Keep password values out of observations/logs. User takeover/manual login remains preferred.
Local-file access	Browser control should reject file:// and other host-local schemes by default unless an explicit future local-file capability is separately sandboxed.
Prompt-injection monitor	Add an independent heuristic/classifier layer only after baseline policy boundaries exist; classifier results can restrict capabilities but must not be the sole defense.
Memory isolation	Do not automatically expose all cross-task memory to web tasks. Retrieve the minimum relevant data and track data provenance/sensitivity.

12.1 Security benchmark gate

Use AgentDojo-style indirect-injection tests plus ST-WebAgentBench policy-compliance tasks and a local BrowserAgent fixture suite. Report both task success and policy success. A safer agent that completes 0% of tasks is not useful; a capable agent that violates policy is not shippable.

13. Completion and Verification

Keep ordinary action verification deterministic. Add task-level verification only at boundaries. The planner must not be allowed to declare “done” solely because its prose sounds plausible.

Layer	Owner	Rule
-------	-------	------

Action verification	existing verifier.py	Did the browser action achieve the expected state/result?
Fact/evidence verification	workspace ingest code	Every important entity attribute/fact must carry source event/evidence. Unsupported fields are not promoted to verified.
Subgoal verification	controller + deterministic predicates where possible	Subgoal completion requires its declared output/evidence conditions, not just an LLM “done”.
Task completion	bounded LLM CompletionEvaluation + deterministic requirement checks	Compare original goal/completion requirements against verified workspace. Reject missing requirements or unsupported claims.

14. Qwen3-8B Feasibility and Inference Budget

Current evidence is stronger than the original research report implied. Qwen3-8B is already validated for schema-constrained action selection, routing, semantic resource references, and long-horizon memory application. The new unknown is high-level general planning, not basic control.

Responsibility	Verdict	Gate
Browser action selection	SUFFICIENT	Existing Phase 1B/5B evidence.
Semantic resource interpretation	SUFFICIENT	Semantic planner holdout already 100% in documented pass.
Workspace structured patches	LIKELY SUFFICIENT	Require schema-validity and factual-evidence benchmark.
2-5 subgoal initial planning	PARTIALLY SUFFICIENT	New unseen-domain planner suite.
Replanning after failures/new constraints	PARTIALLY SUFFICIENT	Controlled failure-injection suite.
Semantic multi-criteria comparison	PARTIALLY SUFFICIENT	Use deterministic numeric preprocessing; compare against hidden labels.
Procedural skill synthesis	UNKNOWN / LATER	Do not implement until controller baseline passes.

14.1 Model-call budget

- Target steady-state direct interaction: ~1 Qwen action call per browser step, matching current design.
- Planner: 1 call at task start, then only on explicit replan boundaries.
- Completion critic: only when controller believes requirements may be satisfied or when budget is nearly exhausted.

- Workspace mutations: where extraction already comes from FinishAction/structured_result, deterministic ingest should avoid an extra model call. Use a workspace-patch call only when semantic normalization is genuinely needed.
- Do not add separate planner model or cloud dependency until benchmarks show qwen3:8b planning is the limiting factor.

15. KEEP / EXTEND / REFACTOR / DEPRECATE Matrix

Component	Decision	Rationale
agent/loop.py	KEEP + EXTEND	Core executor is proven. Add general-subgoal result/parent hooks; avoid rewrite.
agent/schemas.py	EXTEND	Add controller/workspace contracts in separate modules; minimal changes to browser action union.
agent/context_builder.py	EXTEND	Inject current subgoal and top-k relevant workspace entities/evidence under existing token budget.
agent/verifier.py	KEEP + EXTEND	Preserve action verification; add reusable predicates for subgoal conditions where possible.
agent/recovery.py + loop_detector.py	KEEP	Already fixed real dynamic-page loop pathologies; integrate failure signals into controller replan triggers.
browser/page_model.py + observer.py	KEEP initially	Already compact and grounded. Add experimental richer observation behind flag only if benchmark warrants.
browser/playwright_backend.py	KEEP	CDP persistence is a proven product requirement.
memory/event_store.py	KEEP + EXTEND events minimally	Canonical source of truth.
memory/schema.sql	EXTEND	Add workspace projection tables, not a new DB stack.
memory/replay.py	EXTEND	Reconstruct general-controller state/workspace event pointers.
task_memory / FTSS / active_task_facts	KEEP	Proven; do not replace with embeddings.
router/semantic_planner.py	REFACTOR / compatibility	Useful semantic parsing/resource requirements; general controller supersedes fixed execution_shape over time.
router/resources.py	KEEP + EXTEND	Core invariant: model selects real candidate IDs; code owns canonical resources.
BatchOrchestrator	KEEP + EXTEND	Efficient/durable delegate for repeated independent work.

WorkflowOrchestrator	KEEP + EXTEND initially	Validated ordered dependency executor; later benchmark whether controller makes it redundant.
research/discovery.py	KEEP + REFACTOR as capability	General planner may invoke source discovery when evidence is missing.
ui/jobs.py / ui/store.py	EXTEND	Expose current subgoal, controller state, workspace summary, delegation, and clarification without chain-of-thought.
cli/trace.py	EXTEND	Trace plan revisions, workspace mutations, delegate runs, completion evaluations.
legacy/hybrid routing mode	KEEP during migration	Provides rollback and A/B baseline.

16. Minimal New Event Types

Reuse existing events whenever semantics already fit. SUBGOAL_CHANGED already exists and should remain the plan/subgoal event. Add only the following first:

Event	Payload purpose
WORKSPACE_MUTATED	Validated WorkspacePatch + affected projection IDs + source event references.
DELEGATE_STARTED	controller decision, substrate, subgoal, child batch/workflow/task ID.
DELEGATE_RESULT	substrate result summary, verified outputs, child IDs, failure category.
COMPLETION_EVALUATED	satisfied flag, missing requirements, unsupported claims, recommended next step.

Skills, vision, and injection-classifier events should not be added until those phases are actually justified.

17. Configuration and Feature Flags

Extend config/default.yaml conceptually

agent:

control_mode: "legacy" # legacy | general | hybrid

planner_max_subgoals: 5

max_replans: 6

completion_check_after_subgoal: true

max_workspace_entities_in_context: 12

max_workspace_evidence_in_context: 8

security:

default_domain_permission: "browser_control" # read_only | browser_control | no_access

```
block_local_schemes: true
cross_origin_sensitive_transfer_requires_approval: true
```

```
experiments:
  richer_observation: false
  procedural_skills: false
  vision_fallback: false
```

Use the repository's existing AppConfig/dataclass/YAML environment override pattern. Do not introduce another configuration framework.

18. Implementation Phases

Implementation rule: Claude Code must implement exactly one phase at a time on main, run the listed gates, update BROWSERAGENT_MASTER_STATUS.md, and stop. A failed phase is diagnosed before the next phase begins. No “implement the whole redesign” command.

Phase 0 - Baseline Lock and Generality Fixtures

Objective: create evaluation infrastructure before changing architecture. This prevents a new controller from appearing better simply because the tests changed.

- Files: add benchmarks/general_agent/ fixtures and tests/integration/test_general_agent_baseline.py; update docs/BROWSERAGENT_MASTER_STATUS.md.
- Create local deterministic sites for: multi-entity top-3, dynamic DOM churn, cross-site fact dependency, open-tab task, research-source list, prompt-injection page, and a failure/replan scenario.
- Record legacy/hybrid baseline: success, steps, model calls, latency, loop count, recovery transitions, unsupported claims, and safety outcomes.
- PASS: existing 324-test suite remains green; benchmark harness has hidden evaluation criteria and detects at least one known architectural gap (e.g., top-N accumulation) without leaking solution structure.
- DO NOT build workspace/controller in Phase 0.

Phase 1 - Event-Backed TaskWorkspace Projection

Objective: add generic entities/evidence/open questions without changing routing or browser behavior.

- New: agent/workspace_models.py; memory/workspace_store.py.
- Modify: memory/schema.sql, memory/event_store.py (EventType), memory/replay.py only as needed, cli/trace.py, tests/unit/test_workspace_store.py, tests/integration/test_workspace_rebuild.py.
- Reuse task_state.plan/current_subgoal/completed_subgoals and active_task_facts; do not duplicate them in new normalized tables.
- Invariants: events canonical; projection rebuild deterministic; every evidence row references an existing event; no model direct SQL.
- PASS: 100% rebuild equivalence after deleting projection rows; five heterogeneous entity types require zero schema migrations; 1,000 entity/evidence mutations remain within an agreed local latency budget; all existing tests green.
- ROLLBACK: config flag leaves workspace unused; new projection tables are additive.

Phase 2 - General Controller in Shadow/Fixture Mode

Objective: create dynamic task-level planning without replacing AgentLoop or current UI routing.

- New: agent/controller.py, agent/controller_models.py, agent/planner.py.
- Modify: agent/context_builder.py (relevant workspace slice), memory/task_state.py/replay.py for controller fields if necessary, config model/default YAML, trace output.
- Controller uses same qwen3:8b inference client and strict JSON schema. It produces high-level subgoals and bounded replans, never click-level commands.
- Execution in this phase: direct subgoals only, using existing AgentLoop. No batch/workflow delegation yet.
- PASS: unseen local tasks requiring at least 3 distinct procedures achieve $\geq$ legacy success with $\leq$ 25% model-call overhead; planner schema validity 100%; no new hard-coded domain workflow files; crash/restart restores parent plan/workspace.
- FAIL interpretation: if planning lowers success or causes excessive overhead, compare one-shot plan, receding-horizon, and current ReAct-like behavior before proceeding.

Phase 3 - Generic Entity Collection, Evidence, and Top-N Completion

Objective: solve the Amazon-class gap generically.

- New: agent/workspace_ops.py with deterministic query/sort/filter/dedupe/date/numeric operations; optional agent/ranking.py for structured semantic ranking contract.
- Modify: executor finish/result ingest so verified structured findings can create/update generic workspace entities with evidence.
- Benchmark holdouts: vacuum fixture -> laptops -> hotels -> internships -> papers/assignments with no production-code changes.
- PASS Generality Gate: at least four holdout domains solved by the same production code; exactly requested top-k; no unsupported final entity; no task-specific class or workflow added.
- DO NOT add embeddings, vision, or procedural skills.

Phase 4 - Delegation to Existing Batch / Workflow / Research Capabilities

Objective: let the general controller choose efficient proven substrates rather than serially doing everything itself.

- Modify: BatchOrchestrator/WorkflowOrchestrator constructors or wrappers to accept parent_task_id/workspace context and emit delegate result metadata.
- Expose research discovery as controller capability returning real resolved candidate resources.
- Use deterministic selection rules when structure is obvious; model only decides ambiguous strategy.
- PASS: 25/50/100 independent-target tasks retain Phase 5 precision/recall and crash recovery; ordered cross-site dependency preserves verified fact-passing; controller chooses batch for large independent sets and does not regress latency by serializing them.
- Only after this phase may data show that WorkflowOrchestrator is redundant. Until then, do not delete it.

Phase 5 - Completion Verification, UI Integration, and Safety Hardening

Objective: make general mode usable from the existing one-command UI and resistant to injected page instructions.

- Modify: ui/jobs.py, ui/store.py/API response, ui/static/index.html, cli/trace.py, risk/policy layer.
- Display: current subgoal, progress summary, verified findings, delegate activity, waiting-for-input/approval. Do not expose hidden reasoning.

- Security: domain permissions, local-scheme block, cross-origin sensitive-transfer gate, explicit untrusted-content boundaries, injection fixtures.
- Completion: run CompletionEvaluation at task boundaries; deterministic checks reject unmet hard constraints and evidence-free claims.
- PASS: general-mode UI handles stop/restart/clarification; AgentDojo/ST-WebAgentBench-inspired local injections do not cause unauthorized actions/data transfer; consequential action confirmation recall remains at project-defined target; existing UI regression suite green.

Phase 6 - Optional Enhancements Only After Falsification Gates

This phase is not pre-authorized. Implement a component only if earlier telemetry proves the corresponding bottleneck.

Optional component	Build only if...	Do not build if...
Richer AXTree/semantic regions	Current PageObservation misses required controls/text on a statistically meaningful fixture set and richer observation improves success >10% without unacceptable token/latency cost.	Most failures are planning/completion, not observation.
Vision fallback	Visual-only/canvas/layout tasks fail in DOM mode and targeted crops improve success materially on RTX 4070.	VLM cost/VRAM/latency outweighs gains or target tasks are rare.
Embeddings	FTS5/structured skill retrieval misses semantically relevant skills/memory often enough to hurt benchmark success.	Lexical/metadata retrieval remains sufficient.
Procedural skills	General controller already succeeds; repeated tasks show >=25-30% step/token savings with low negative transfer.	Skills reduce reliability or merely cache brittle website flows.
Parallel agents	Batch throughput is inadequate and model/browser contention tests show real speedup.	Single local model becomes the bottleneck or site stability declines.
Larger/separate planner model	Qwen planner is a measured dominant error source after prompt/schema fixes.	Errors remain interface/verifier/resource bugs.

19. Generalization Benchmark Specification

Category	Hidden success criteria	Key metrics
Unknown-procedure	Goal achieved without predefined workflow-specific code.	Task success, planner revisions, unsupported actions.
Multi-entity / top-N	Collect >=N candidates, evidence-backed attributes, return exact k correct selections.	Entity completeness, evidence coverage, ranking correctness.
Long horizon	Complete 50-200 action task without context explosion/looping.	Success, loop rate, prompt tokens, recovery transitions.
Cross-site dependency	Use exact verified value from earlier source	Value accuracy, provenance, contradiction

	at later destination.	guard.
Dynamic DOM	Operate despite irrelevant page churn.	Repeat-action rate, false verifier failures.
Research	Gather diverse sources, evidence-backed synthesis, know when to stop.	Source diversity, unsupported claims, completion precision.
Resource ambiguity	Select correct open tab/resource or ask user.	Resolution accuracy, clarification precision, URL hallucinations.
Failure/replan	Recover from blocked path/site, choose alternate strategy.	Replan success, wasted actions.
Crash/resume	Kill process at planner/action/workspace boundaries.	Lost/duplicate actions, workspace reconstruction.
Prompt injection	Malicious page content attempts to alter task/exfiltrate data.	Policy success, task success under attack, risk ratio.

20. Trace and Observability Additions

Extend browser-agent trace rather than create a second diagnostics system. New concise event renderings should include: parent task ID, planner/controller decision reason_code, plan/subgoal transitions, workspace mutations (entity IDs/field names, not secret values), evidence source event IDs, delegate start/result child IDs, completion evaluations, and per-phase model/action latency. Never log chain-of-thought or passwords/cookies.

21. Risks and Mitigations

Risk	Why it matters	Mitigation
Planner becomes a new brittle bottleneck	A general controller can fail before the proven executor begins.	Hybrid feature flag, schema contract, bounded replanning, legacy baseline, holdouts.
Workspace grows without bound	Large candidate sets can recreate context explosion.	Persist all; retrieve only task-relevant top-k entities/evidence; deterministic summaries.
General mode serializes scalable work	Naive controller might inspect 100 sites one by one.	Keep BatchOrchestrator as delegate and add deterministic substrate selection.
Prompt injection exploits broader autonomy	Controller can access more capabilities/data.	Least privilege, domain/data-flow policy, approvals, untrusted-content separation, security benchmark.
Premature finish	Planner may infer success from partial evidence.	Completion requirements + evidence provenance + boundary critic + deterministic hard constraints.
Overengineering skills/vision/embeddings	New components add failure modes before baseline generality works.	Explicit falsification gates; Phase 6 only.

Model-size escalation masks architecture bugs	Prior failures often came from interfaces/evaluators.	Error taxonomy and ablations before changing Qwen.
---	---	--

22. Claude Code Handoff Contract

Instruction to Claude Code: Read the current repository and this document. Treat repository main @ 58a7d5a as the baseline. Implement only the phase explicitly requested by the user. Do not invent a parallel architecture, do not delete validated subsystems, do not start later phases early, and do not weaken deterministic verification/safety to make a benchmark pass.

22.1 Repository invariants Claude must preserve

- Event log remains canonical; materialized state/workspace must be rebuildable.
- The model never receives or caches raw Playwright handles. Element IDs are observation-scoped and re-resolved before action.
- The model cannot invent canonical resource URLs/tabs when deterministic enumeration exists.
- Browser action output remains schema constrained and semantically validated.
- Consequential actions remain approval-gated independently of planner claims.
- Crash recovery never blindly repeats an ambiguous consequential action.
- Context remains bounded; persistent state lives in SQLite, not hidden model history.
- No Amazon/product/course/site-specific production conditionals are allowed to satisfy generality tests.
- Existing Batch/Workflow/Research behavior remains available during migration unless a later benchmark explicitly proves removal safe.

22.2 Recommended first Claude request

Implement Phase 0 only from the BrowserAgent General Autonomous Agent Architecture document. Work directly on current main. Do not implement TaskWorkspace or the general controller yet. Create the hidden-evaluation local generality benchmark/fixture harness, capture the legacy baseline, run the complete existing suite, update docs/BROWSERAGENT_MASTER_STATUS.md, commit and push main, then stop and return the exact baseline metrics and any benchmark-design issues discovered.

23. Factual Source Notes and Bibliography

Repository claims in this document were checked against BrowserAgent main @ 58a7d5a095900be07614e2992d5c14b72732036c, including docs/BROWSERAGENT_MASTER_STATUS.md, agent/schemas.py, agent/loop.py, browser/page_model.py, browser/observer.py, memory/event_store.py, memory/schema.sql, memory/replay.py, router/semantic_planner.py, router/plan_schema.py, batch/orchestrator.py, and workflow/orchestrator.py.

[1] OpenAI - Introducing Operator. <https://openai.com/index/introducing-operator/>

[2] OpenAI - Computer-Using Agent. <https://openai.com/index/computer-using-agent/>

[3] OpenAI - Introducing ChatGPT agent: bridging research and action. <https://openai.com/index/introducing-chatgpt-agent/>

[4] OpenAI - ChatGPT agent System Card. <https://openai.com/index/chatgpt-agent-system-card/>

[5] OpenAI Help - ChatGPT agent release notes / Operator integration.

<https://help.openai.com/en/articles/11794368>

[6] OpenAI Help - Atlas deprecation / browser capabilities moving to ChatGPT.

<https://help.openai.com/en/articles/20001371-evolving-atlas-into-chatgpt-for-browser-based-agentic-work>

[7] OpenAI Help - Cloud browser in ChatGPT Work. <https://help.openai.com/en/articles/20001280-using-cloud-browser-in-chatgpt>

[8] OpenAI - Designing AI agents to resist prompt injection. <https://openai.com/index/designing-agents-to-resist-prompt-injection/>

[9] Perplexity - Comet official product page. <https://www.perplexity.ai/comet>

[10] Perplexity Help - Assistant Panel. <https://www.perplexity.ai/help-center/comet/en/articles/11734688-assistant-panel>

[11] Perplexity Help - Managing Comet Assistant permissions.

<https://www.perplexity.ai/help-center/en/articles/13531023-managing-comet-assistant-permissions>

[12] Perplexity Help - Comet Assistant Privacy & Data Use.

<https://www.perplexity.ai/help-center/comet/en/articles/12867415-comet-assistant-privacy-data-use>

[13] Anthropic - Developing a computer use model. <https://www.anthropic.com/news/developing-computer-use>

[14] Anthropic - Mitigating the risk of prompt injections in browser use.

<https://www.anthropic.com/news/prompt-injection-defenses>

[15] BrowserGym ecosystem paper. <https://arxiv.org/abs/2412.05467>

[16] Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks.

<https://arxiv.org/abs/2503.09572>

[17] AgentDojo: prompt injection evaluation environment. <https://arxiv.org/abs/2406.13352>

[18] ST-WebAgentBench: safety/trustworthiness benchmark. <https://arxiv.org/abs/2410.06703>

[19] Assessing Automated Prompt Injection Attacks in Agentic Environments (2026).

<https://arxiv.org/abs/2606.10525>

[20] browser-use open-source project. <https://github.com/browser-use/browser-use>

24. Final Implementation Verdict

The prior research direction is retained but narrowed to what the repository and external evidence support. The correct next move is not another architecture rewrite and not another specialized Amazon fix. It is to lock a generality benchmark, add an event-backed generic workspace, add a low-frequency dynamic controller around the existing AgentLoop, prove generic multi-entity reasoning, then let that controller delegate to the already validated batch/workflow/research machinery. Skills, embeddings, vision, parallel multi-agent execution, and larger planner models remain optional experiments behind explicit falsification gates.

Ready-to-feed status: YES. This revised document is intended to be fed to Claude Code, but Claude should implement only the explicitly requested phase. The recommended next prompt is Phase 0 baseline lock, not “implement everything.”